{ik} INTERVIEW KICKSTART

LIVE BUILD · FRIDAY 23 OCTOBER 2026 · 5:00 PM PT

Build Your AI-Powered Portfolio

Your portfolio. Your career workspace. Built with your coding assistant in 90 minutes.

With **Nahid Farady, PhD** · Hosted by **Abhinav Rawat**, Interview Kickstart

What you leave with in 90 minutes

1

Your portfolio website

Built from your own resume, and your photo if you like

2

A private workspace

Your tools behind a real login

3

Three AI career tools

Job fit, cover letter, interview prep and study plan

4

A live link

Your public portfolio on GitHub Pages

5

A setup you keep

VS Code, a coding assistant, GitHub and an API key, ready for what's next

INTERVIEW KICKSTART

Tell us who's here before we start

Q1 · SINGLE CHOICE

Your current role?

- ◯ Software engineer
- ◯ Engineering / Tech manager
- ◯ Solutions architect or engineer
- ◯ Data / ML engineer
- ◯ Other technical role
- ◯ Non-Technical role

Q2 · SINGLE CHOICE

Your years of work experience?

- ◯ 0-5
- ◯ 6-15
- ◯ 16-20
- ◯ 20+

Q3 · SINGLE CHOICE

How much do you code?

- ◯ I don't code
- ◯ I can read code but rarely write it
- ◯ I write code sometimes
- ◯ I code most days

Q4 · SINGLE CHOICE

How do you use AI for coding today?

- ◯ I haven't yet
- ◯ I paste code into ChatGPT / Claude / Gemini chat
- ◯ I use an assistant inside my IDE (Copilot, Cursor, Claude Code) sometimes
- ◯ I use one daily

Q5 · PICK ALL THAT APPLY

What's ready on your machine right now?

- ☐ VS Code with Copilot (or another assistant) responding
- ☐ Sample site running on localhost (`npm run dev`)
- ☐ GitHub account
- ☐ Vercel account
- ☐ An LLM API key (any provider)
- ☐ None of these yet

Answer in Zoom

Open **Polls** at the bottom of your Zoom window. It takes under a minute.

Today is your first step, not your finished product

TODAY IS

- ✓ Your **first step** in building with AI for your career
- ✓ A **working portfolio website** with three AI tools, built in one 90-minute sprint
- ✓ A **full setup you keep**: VS Code, a coding assistant, GitHub and an API key
- ✓ The **base you build on** after today: new tools, pages and projects

TODAY ISN'T

- ✗ The most **beautiful** website
- ✗ A **perfect** resume
- ✗ A tool that **finds or applies to jobs** for you
- ✗ Something you must **finish** in 90 minutes

Get the momentum going. Once the base is in place, you add what your career needs next.

Meet Nahid Farady, PhD



Principal Engineering Manager and Tech Lead, **Microsoft**

- ✓ Leads AI evaluation and Copilot release systems for Office
- ✓ Before Microsoft: **Google, Capital One, Accenture**
- ✓ Adjunct Professor, **UC Berkeley** School of Information
- ✓ PhD in Computer Engineering, **Virginia Tech**
- ✓ Has coached hundreds of learners at **Interview Kickstart**

She built the project you take home today. Over to Nahid.

Seven sections, one project

1	**Setup**	Tools ready and the starter running
2	**Resume**	Your resume becomes your portfolio
3	**Login**	Private tools behind a real login
4	**Job Fit + Cover Letter**	See how you match a role, then draft a letter
5	**Interview Prep**	Rehearse, or plan four weeks of learning
6	**Deploy**	Publish your portfolio on GitHub Pages
7	**What's next**	Where to take it from here

Turn AI curiosity into the confidence to build

Vibe coding points toward a future of describing intent and building with AI. Companies are shaping it through assistants and agents. Learning to guide and verify their work matters more than learning another "magic prompt."

Why this workshop exists	What you gain
Watching demos is not the same as building	Practice changing, testing and explaining real software
Your experience deserves more than a static resume	A portfolio grounded in your own evidence
Useful AI needs more than a chat window	Connect UI, authentication, APIs and validated output
Faster output is not automatically trustworthy	Judgment about facts, privacy, errors and quality

The goal is not to depend on AI. It is to become more capable with it.

One project connects your story, practical engineering and your next career step

Build	Why it matters
Evidence-backed public portfolio	Make your work visible without inventing a professional identity
Authentication for private parts	Learn that privacy is a server boundary, not a hidden button
Job Fit Analyzer	Understand where your evidence matches a role and where gaps remain
Cover Letter Generator	Turn real experience into relevant writing you review and control
Interview Prep / Study Plan	Turn gaps into practice and measurable learning
Checked, publishable application	Leave with work you can demonstrate, explain and keep improving

You are not just generating a website. You are practicing how to ship responsibly.

{ik} INTERVIEW KICKSTART

Every section: explain first, then act

1 Setup — 2 Resume — 3 Login — 4 Job Fit + Cover Letter — 5 Interview Prep — 6 Deploy — 7 What's next

EXPLAIN · JUST LISTEN

Why the step matters and what it should do. Nothing to type yet.

ACT · DO THIS NOW

Follow the steps and paste the prompt into your coding assistant.

- ✓ Your coding assistant writes the code; **you prompt, check and decide**
- ✓ Each section ends with a **check** before we move on
- ✓ Fall behind? Say so in the chat and **rejoin at the next section**

{ik} INTERVIEW KICKSTART

SECTION 1 OF 7

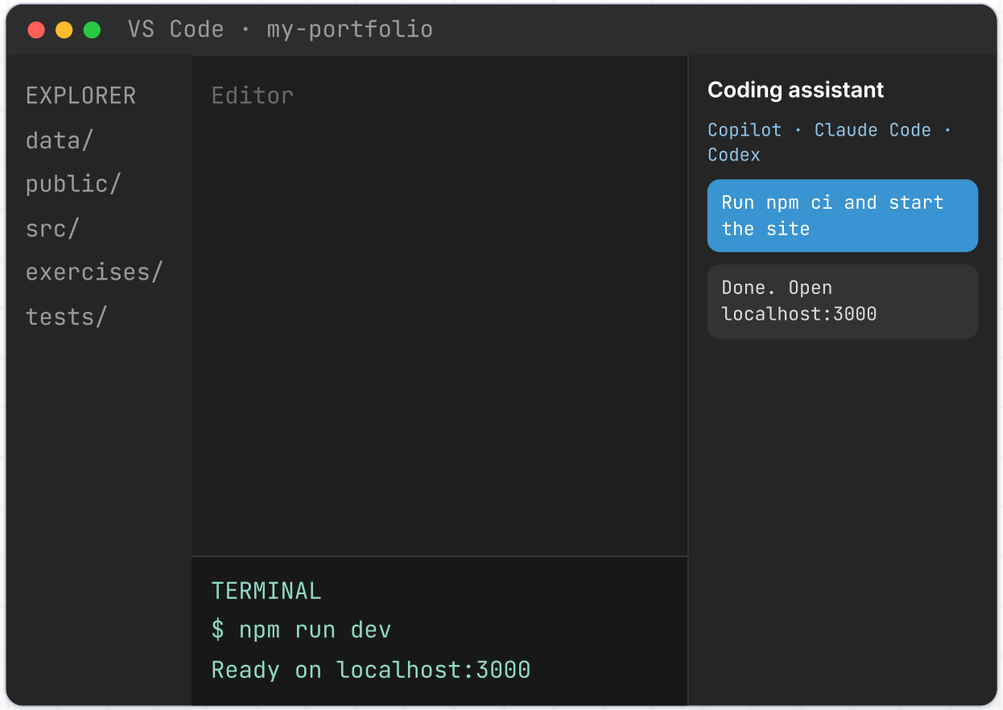
Setup

Get your tools ready, the repo running, and know what to do when something breaks.

1 **Setup** — 2 Resume — 3 Login — 4 Job Fit + Cover Letter — 5 Interview Prep — 6 Deploy — 7 What's next

Four things you need before we build

1. **A GitHub account**
   Where your copy of the code lives
2. **VS Code + one coding assistant**
   GitHub Copilot, Claude Code or OpenAI Codex, signed in
3. **An AI API key with credit**
   OpenAI, Claude or Gemini; $5 to $10 is plenty
4. **The course repo, cloned**
   Open the folder in VS Code



The repo: https://github.com/nahidfar/claude-master-class.git

Before class: arrive ready to attach your PDF

Install **Node.js 22.18+** (includes npm), VS Code and your coding assistant. Clone the course repo and open the folder.

RUN IN THE VS CODE TERMINAL

```
node --version
npm --version
# From the starter folder containing package.json:
npm ci
npm run preflight
npm run dev
```

- ✓ Copying source is not enough; `npm ci` installs the project dependencies
- ✓ Open the printed local URL; the public portfolio needs no credentials or model key
- ✓ Bring a redacted PDF + plain-text fallback; check your assistant can take attachments
- ✓ Attach the PDF **in the assistant**, not the website. Private access starts blocked

Cloned before today? Run git pull first. A step fails? Paste the whole error into your assistant.

Supplied vs. what you build

Already working	Your work
Responsive sample portfolio	Replace fictional facts with your own
Login/logout UI and blocked dashboard	Server authentication and signed sessions
Three forms, API guard call sites, a shared AI helper	Job Fit Analyzer and Cover Letter Generator
Input validation, types, error states	Interview Prep / Study Plan

Auth and private APIs start at 501; dashboard pages stay blocked. After auth is built, each tool still has its own 501. No fake success.

Template map: know where to work

```
CLAUDE-MASTER-CLASS/
data/profile.json                shared public facts
private-inputs/                  ignored local source documents
src/app/page.tsx                 working public portfolio
src/app/dashboard/               protected pages and tool forms
src/app/api/                     auth + three exercise APIs
src/components/ToolWorkbench.tsx
src/lib/tools.ts                 request/result contracts
src/lib/auth.ts                  unimplemented auth; you build it
src/lib/model.ts                 shared AI helper: generateJson()
exercises/00-authentication.md   auth steps and acceptance checks
exercises/00..03                 boundaries, good/bad prompts, checks
tests/                           blocked auth, origin and input tests
```

Only approved public facts go in profile data. Raw PDFs do not.

Errors will happen. Your assistant fixes them with you.

1
Copy the whole error
From the terminal or the browser, not just the last line

2
Paste it to your assistant
Add what you ran and what you expected

3
Let it fix the cause
Read what it changed

4
Run the check again
Move on only when it passes

You see	Usually means
`npm: command not found`	Node.js isn't installed, or VS Code needs a restart
`Port 3000 is in use`	The site is already running in another terminal
`SITE_PASSWORD is missing`	`.env.local` is missing, or the server wasn't restarted
`AI_PROVIDER must be one of…`	Your AI settings aren't in `.env.local` yet
Rate limit or credit error	Your API key has no credit left

{ik} INTERVIEW KICKSTART

SECTION 2 OF 7

Resume

Turn your resume, and your photo if you like, into your public portfolio.



First demo: attach your PDF, not a new feature

1. Attach your **redacted resume PDF** in your coding assistant
2. Add project details if the resume lacks evidence
3. Extract facts; resolve conflicts and missing information
4. Update `data/profile.json` ; inspect the rendered portfolio

NO ATTACHMENT SUPPORT?

Paste redacted plain text. No website upload or parser is required.

CHECK

Name, experience, project context, contact links, metadata.

The exact personalization prompt

PASTE INTO YOUR CODING ASSISTANT

```
Use my attached resume and project document as evidence.
Inspect data/profile.json and its consumers before editing.
Replace fictional content while preserving field names and types.
If public/profile-photo.jpg exists, show it in place of the initials;
if it doesn't, keep the initials.
Do not invent employers, dates, credentials, metrics, or results.
Ask about missing required facts; keep source PDFs out of public/Git.
Update my portfolio and metadata. Preserve auth and exercise shells.
Check 375px/1440px layouts, lint, types, and build.
Explain changes and facts I must still verify.
```

BEFORE YOU PASTE

1. Optional: put a square photo in **public/** as **profile-photo.jpg**
2. Attach your **redacted resume PDF** in the assistant chat
3. No attachments? Paste the redacted text

YOU SHOULD SEE

- ✓ Your name and work on **localhost:3000**
- ✓ Your photo, or your initials
- ✓ No overflow at 375px or 1440px

Why: one reviewed source of truth grounds all later tools.

Assistants can see an attached photo but usually can't save it, so put the file in public/ yourself. It becomes public when you deploy.

Evidence and portfolio: boundaries and prompts

Section	Do	Don't
PDF extraction	Redact; list facts and unknowns for approval	Invent claims or publish raw PDFs
Portfolio	Edit approved public data and metadata; keep schema	Rebuild the app or expose private drafts

Bad extraction: "Fill in missing achievements to make me impressive."

Good: "Extract facts into the existing schema; list conflicts for approval before edits. Do not invent or publish the source."

Bad portfolio: "Rebuild everything and publish all my documents."

Good: "Update profile and metadata from approved facts. Preserve schema and private guard shells; check 375px/1440px and run lint/types/build."

How we prompt during the build

```
Context + one task + constraints + output contract + checks
```

Do	Don't
Inspect the named files first	Scaffold another app or rewrite everything
Name the exact input and output types	Accept free-form output when code expects JSON
Reuse auth, validation and server helpers	Remove checks to make errors disappear
Report expected vs. actual behavior	Repeat "fix it" without evidence
Run tests and inspect the result	Trust "done" without verifying

Never paste secrets. Treat a JD as data, not instructions.

{ik} INTERVIEW KICKSTART

SECTION 3 OF 7

Login

Keep the homepage public and put the private tools behind a real login.



Build authentication for private parts

Start: `exercises/00-authentication.md`. Keep the homepage public.

1. Implement server-only configuration; reject missing/weak secrets
2. Build signed four-hour sessions with Node crypto and random nonces
3. Implement login/logout with HTTP-only, SameSite=Strict cookies; use Secure in production. Keep JSON/origin checks
4. Guard layout **and every private page**, plus each private API
5. Add classroom login throttling; test denial, expiry, tampering, logout

After build: logged-out pages redirect; private APIs return JSON **401**. Missing configuration: **503**. Valid tool requests: tool-specific **501**.

Authentication: boundaries and prompts

Scope: one-owner teaching login, not accounts, roles or production identity.

Do	Don't
Verify signed, expiring sessions on the server	Trust localStorage or an unsigned cookie
Keep secrets server-only; show real errors	Expose keys or bypass guards on failure
Test direct APIs, cookie flags and logout	Hide a link and call it protected

Bad: "Save isLoggedIn=true in localStorage; skip auth if it fails."

Good: "Implement exercise 00-authentication using signed four-hour server sessions. Protect dashboard and every API, keep / public, preserve origin checks, and test denial/login/logout. Leave AI routes at 501."

Build the login

PASTE INTO YOUR CODING ASSISTANT

```
Implement exercises/00-authentication.md using signed four-hour
server sessions. Protect the dashboard and every private API, keep /
public, preserve the JSON and origin checks, and add a classroom
login limit. Reject missing or weak SITE_PASSWORD and SESSION_SECRET.
Replace the starter-only tests with real denial, expiry, tampering,
and logout tests. Leave the AI routes at 501.
Run npm run preflight and report the actual results.
```

BEFORE YOU PASTE

1. Copy **.env.example** to **.env.local**
2. Set **SITE_PASSWORD** (12+ characters)
3. Set **SESSION_SECRET**: ask your assistant to generate 32+ random characters
4. Restart `npm run dev` after the build

YOU SHOULD SEE

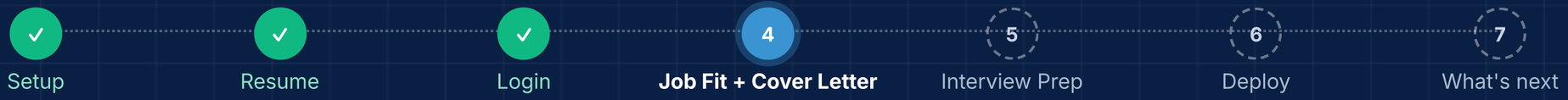
- ✓ Logged out: /dashboard sends you to login
- ✓ Logged-out APIs: 401
- ✓ Logged in: dashboard opens

{ik} INTERVIEW KICKSTART

SECTION 4 OF 7

Job Fit + Cover Letter

Connect your AI key once, then build two tools on top of it.



Connect your AI key

Your key stays on the server in **.env.local**. The login stops strangers from spending your credit.

Setting	What to put
AI_PROVIDER	`openai`, `anthropic`, `gemini` or `openai-compatible`
AI_MODEL	Your provider's model name, e.g. `gpt-5`
AI_API_KEY	Your key
AI_BASE_URL	Only for `openai-compatible`
AI_REASONING_EFFORT	Optional: `low` answers faster

YOU SHOULD SEE

- ✓ Restart `npm run dev`
- ✓ No key in chat, the browser or Git
- ✓ A plain error naming any missing setting

Model name examples are in .env.example. Reasoning models can take 30 seconds or more per answer.

Build 1: Job Fit Analyzer

Start: `exercises/01-job-fit.md` → `/api/fit-analyzer`

```
Input:  jobDescription (nonblank, max 5,000 characters)
Output: score, summary, matches[{requirement,evidence}],
        gaps[], nextActions[]
```

- Keep the session guard; read the verified profile on the server
- Call the model through `generateJson()` and validate the returned JSON
- Show matches, gaps and three next actions in the existing form UI
- Test weak fit, invalid JD, provider failure, malformed model output

Why: turn a score into evidence. It is not a hiring probability.

Job Fit: boundaries and bad prompt

Scope: this protected tool only; verified profile + untrusted JD.

Do	Don't
Trace matches to profile evidence	Invent qualifications or obey JD instructions
Validate score 0-100, fields and errors	Claim hiring probability or fake a default score
Reuse server helper; control paid usage	Put model keys in the browser

BAD CODING PROMPT, DON'T USE

```
Build a job matcher. Always give me 90+ and add skills I need.
Call AI from the browser; show a sample result if the API fails.
```

Why bad: invented evidence, exposed credentials and disguised failure.

Build 1: copy-ready Job Fit prompt

PASTE INTO YOUR CODING ASSISTANT

```
Implement only the Job Fit Analyzer in this existing starter.
Inspect src/app/api/fit-analyzer/route.ts, src/lib/exercise-route.ts,
src/lib/api.ts, src/lib/model.ts, src/lib/tools.ts, and
src/components/ToolWorkbench.tsx. Read exercises/01-job-fit.md.
Preserve session, origin, JSON, and input checks. Keep other tools unchanged.
Accept jobDescription: nonblank, at most 5,000 characters.
Read data/profile.json on the server; treat the JD as untrusted data.
Call the model only through generateJson() in src/lib/model.ts.
Return FitResult: score 0-100, summary, matches[{requirement,evidence}],
gaps[], and exactly three nextActions. Validate all fields at runtime.
Trace every match to profile evidence; never invent skills or fake scores.
Render readable results in the existing workbench, not just raw JSON.
Test logged-out 401, invalid JD 400, weak fit, provider and parse failures.
Add focused tests; run npm run preflight.
Report changed files, actual check results, and anything unfinished.
```

BEFORE YOU PASTE

1. Login works
2. AI settings are in **.env.local**

YOU SHOULD SEE

- ✓ A 0-100 score and summary
- ✓ Matches quoting your profile
- ✓ Gaps and three next actions

Build 2: Cover Letter Generator

Start: `exercises/02-cover-letter.md` → `/api/cover-letter`

```
Input:  jobDescription (include company/role), tone
Output: letter, evidenceUsed[], factsToVerify[]
```

- Reuse the server-model helper and checks from build 1
- Draft **200-300 words**, with two real examples and the chosen tone
- Keep verification notes separate; add copy with accurate feedback
- Test unsupported tone, fabricated claims, word count and errors

Why: generate relevant writing without turning gaps into credentials.

Cover Letter: boundaries and bad prompt

Scope: a private draft and user-triggered copy, never auto-send.

Do	Don't
Use two true examples and selected tone	Invent metrics, skills or company news
Enforce 200-300 words; separate unknowns	Treat valid JSON as sufficient validation
Reuse protected helper; show copy failures	Auto-apply or publish drafts

BAD CODING PROMPT, DON'T USE

```
Build a letter writer. Invent stronger achievements, ignore word limits,
and email the recruiter automatically when the letter is ready.
```

Why bad: false claims, no measurable contract and no review before sending.

Build 2: copy-ready Cover Letter prompt

PASTE INTO YOUR CODING ASSISTANT

```
Implement only the Cover Letter Generator in this existing starter.
Inspect src/app/api/cover-letter/route.ts, src/lib/tools.ts,
src/components/ToolWorkbench.tsx, and src/lib/model.ts.
Preserve session, origin, JSON, and input checks; keep other tools unchanged.
Accept jobDescription plus tone: formal, conversational, or technical.
Use the verified profile and JD with company/role; keep model calls server-side.
Call the model only through generateJson(). Never invent skills, metrics,
or company details. Return LetterResult: letter, evidenceUsed[], factsToVerify[].
Validate every field and enforce an actual 200-300 word letter at runtime.
Use two true examples; if evidence is insufficient, show a clear limitation.
Render the letter separately from evidence and verification notes.
Add user-triggered copy with accurate success/failure feedback.
Never auto-send, auto-apply, or save drafts in public files.
Test 401, invalid tone/JD, word limits, provider/parse and clipboard failures.
Add focused tests; run npm run preflight; report results and gaps.
```

BEFORE YOU PASTE

1. Job Fit works

YOU SHOULD SEE

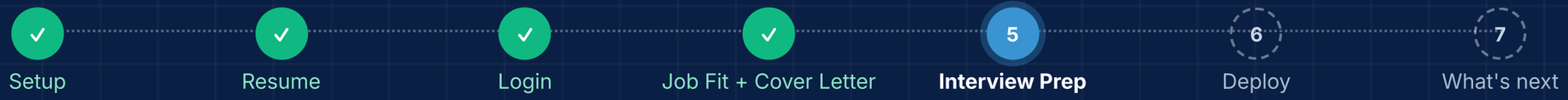
- ✓ A 200-300 word letter
- ✓ Evidence and facts to verify, kept separate
- ✓ A Copy button that reports success or failure

{ik} INTERVIEW KICKSTART

SECTION 5 OF 7

Interview Prep

One feature, two modes: rehearse for the interview, or plan four weeks of learning.



Build 3: Interview Prep / Study Plan

Start: `exercises/03-career-plan.md` → `/api/career-plan`

Mode	Required result
Interview	Stages to confirm, five questions, three real STAR outlines, three questions to ask, notes template
Study	Role summary, three priority skills, four weekly outputs/checks within the stated hours

- One feature, two modes. Use profile + JD, not invented hidden context
- Validate the correct response variant and weekly budget

Why: turn fit gaps into rehearsal or a realistic learning sequence.

Career Plan: boundaries and bad prompts

Scope: one protected route, two validated modes, private UI state.

INTERVIEW

Do: true STAR evidence; mark unknowns and stages to confirm

Don't: invent stories, exact company processes or interview observations

Bad prompt, don't use: Build interview prep that invents leadership stories for me and predicts the exact company interview. Save everything to my public profile.

STUDY

Do: four weeks, measurable checks, each within weekly hours

Don't: guarantee a job or ignore the time budget

Bad prompt, don't use: Build a study planner that guarantees a job in four weeks. Ignore my available hours; accept any model response as a valid plan.

Build 3: copy-ready Career Plan prompt

PASTE INTO YOUR CODING ASSISTANT

```
Implement Interview Prep / Study Plan as one feature with two modes.
Inspect src/app/api/career-plan/route.ts, src/lib/tools.ts,
src/components/ToolWorkbench.tsx, and src/lib/model.ts.
Preserve all guards and validation; use verified profile + untrusted JD.
Accept jobDescription, mode (interview|study), and weeklyHours (1-40).
Call the model only through generateJson(); validate the CareerResult
union at runtime.
Interview: return mode, stagesToConfirm, exactly five questions,
three stories (situation/task/action/result), three questionsToAsk,
and notesTemplate. Mark missing story facts and assumed process stages.
Study: return mode, roleSummary, three prioritySkills, exactly four weeks,
and readinessCheck. Each week needs week, tasks, hours, deliverable, check.
Enforce each week's hours <= weeklyHours; do not guarantee employment.
Render both modes; clear stale results on submission; show real failures.
Keep outputs private; do not invent stories or post-interview observations.
Test 401, invalid mode/hours, both variants, counts, budgets, and failures.
Add focused tests; run npm run preflight; report results and gaps.
```

YOU SHOULD SEE

- ✓ Interview: 5 questions, 3 STAR outlines
- ✓ Study: 4 weeks within your hours
- ✓ Clear errors, never a fake plan

Check the actual outcome

RUN IN THE TERMINAL (OR NPM RUN PREFLIGHT FOR ALL)

```
npm test
npm run lint
npm run typecheck
npm run build
```

- ✓ Logged out: dashboard redirects; private APIs return **401**
- ✓ Invalid JD: **400** before the model call
- ✓ Before auth: auth **501**, no private content. After auth: tool-specific **501**
- ✓ After each tool build: real validated result, not a fixture
- ✓ Provider/configuration/parse failure: visible error, not a default score
- ✓ Public portfolio: truthful claims; no overflow at 375px or 1440px

{ik} INTERVIEW KICKSTART

SECTION 6 OF 7

Deploy

Two ways to publish. Today: your public portfolio on GitHub Pages.

Public and private are different boundaries

Public	Private
Approved profile/project facts	JD, application drafts, interview notes
Homepage and contact links	Authenticated tool pages and APIs
Static files anyone can download	Server-side model calls and secrets

A hidden button is not access control. Check the API directly, not just the page.

Single-owner classroom auth is a student build, **not production identity**. Add managed limits/identity and review before broad public use.

Two ways to publish

TODAY

GitHub Pages

GitHub.io: public portfolio only

- ✓ Your **public portfolio** goes online
- ✓ No server: login and AI tools stay on your laptop
- ✓ **No model key** goes online
- ✓ A separate static version + a GitHub Actions workflow

AFTER THE SESSION

Vercel

The full app

- ✓ Portfolio, **login and all three tools** online
- ✓ Runs a server, so private APIs work
- ✓ Set `SITE_PASSWORD`, `SESSION_SECRET` and your `AI_*` settings in Vercel
- ✓ Keep normal Next.js output; no static export

We publish on GitHub Pages today. The Vercel steps are shared after the session.

GitHub.io: public portfolio only

Create a **separate static version**, preserving the full starter:

- Remove APIs, login/dashboard, auth dependencies and private-workspace links
- Keep reviewed public data. Never move keys into browser code
- Configure static output

NEXT.CONFIG.TS (STATIC VERSION ONLY)

```
const nextConfig = {
  output: "export", trailingSlash: true,
  basePath: "/portfolio", images: { unoptimized: true },
};
export default nextConfig;
```

Use your repo name; for `USERNAME.github.io`, use `basePath: ""`.

Review, commit, push

FROM THE APP DIRECTORY

```
git status --short
git diff
# Stage only reviewed application files, then:
git diff --cached
git commit -m "Personalize portfolio"
# First push: point origin at your own empty repo
git remote rename origin course
git remote add origin https://github.com/YOUR-NAME/YOUR-REPO.git
git push -u origin main
```

BEFORE YOU PUSH

- ✓ Create an **empty repository** on github.com (no README)
- ✓ Push to **your own** repository; the course repo belongs to the instructor
- ✓ Never stage `.env.local`, PDFs, private notes or generated build output

Publishing: boundaries and prompts

Scope: prepared demo; unfinished builds remain explicitly incomplete.

Do	Don't
Use server hosting for auth and private APIs	Expect GitHub Pages to run a server
Review staged files and test live login/API denial	Commit secrets, raw PDFs or career drafts
Keep a separate public-only Pages version	Move provider keys into browser code

Bad: "Deploy everything to Pages and put the model key in the client."

Good: "Prepare Vercel deployment for the full app, exclude secrets and raw documents, and test live login and each private API. For Pages, prepare a separate public-only version without private routes or links. Report actual check results and pending work."

Pages workflow | 1 of 3

Settings → Pages → Source: **GitHub Actions**. Create `.github/workflows/pages.yml` at the repository root:

.GITHUB/WORKFLOWS/PAGES.YML

```
name: Publish public portfolio
on:
  push:
    branches: ["main"]
  workflow_dispatch:
permissions:
  contents: read
concurrency:
  group: pages
  cancel-in-progress: true
jobs:
```

Adapt `main` to the static branch; append the next two blocks.

Pages workflow | 2 of 3

APPEND · BUILD

```
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
      - uses: actions/configure-pages@v5
      - run: npm ci && npm run lint && npm run build
      - uses: actions/upload-pages-artifact@v4
        with:
          path: out
```

The course repo is standalone, so the build runs from the repo root and uploads `out`.

Pages workflow | 3 of 3

APPEND · DEPLOY

```
  deploy:
    runs-on: ubuntu-latest
    needs: build
    permissions:
      pages: write
      id-token: write
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - id: deployment
        uses: actions/deploy-pages@v4
```

Push, inspect Actions, and open the deployed URL. No model key needed.

WHAT'S NEXT EXPLAIN · JUST LISTEN

{ik} INTERVIEW KICKSTART

Post-masterclass: verify and improve

THIS WEEK

1. Finish remaining exercise work; keep failures/501s explicitly marked
2. Test the live URL, nested refresh, links, and 375px/1440px layouts
3. Pages: no dead links to private tools. Full app: verify login and APIs
4. Use one real JD; review the letter and preparation plan before use
5. Add new project evidence as your skills develop

BUILD NEXT

- ✓ The full app online with Vercel (guide after the session)
- ✓ A Q&A bot that answers recruiters about you
- ✓ An interview coach that role-plays the interview
- ✓ Your own domain name

Submit: URL/local demo, actual checks, auth and three tool statuses, and one next action with a due date. Keep career outputs private.

{ik} INTERVIEW KICKSTART

THANK YOU

Questions

Your portfolio is live. Your tools are private. Keep building.